CENTRO UNIVERSITÁRIO DE BRASÍLIA — UniCEUB

Faculdade de Tecnologia e Ciências Sociais Aplicadas

Curso de Ciência de Dados

MOTOAR

Plataforma de Qualidade do Ar para Motociclistas de Brasília

Planejamento Arquitetural — Parte 1

Alessandra Ferreira — Matrícula: 22304883
Luiz Henrique — Matrícula: 22304604
Rafael — Matrícula: 22304454

Disciplina: Engenharia de Dados
Professor: Luis Carlos Cardoso
Repositório: github.com/alessandra2307/Projeto-egenharia-de-dados

Brasília, 2026

RESUMO

O projeto MotoAR tem como objetivo desenvolver uma plataforma de dados capaz de coletar, processar e analisar informações de qualidade do ar e condições meteorológicas em Brasília, gerando recomendações práticas e personalizadas para motociclistas. A motivação parte de uma lacuna real: Brasília concentra mais de 500 mil motocicletas, mas nenhuma ferramenta pública e acessível integra dados de poluição atmosférica com recomendações de equipamento de proteção individual.

Este relatório documenta o planejamento arquitetural completo da Parte 1 do projeto, cobrindo a identificação e classificação dos dados, os domínios e serviços de negócio envolvidos, a arquitetura de dados escolhida (Arquitetura Medalhão com Delta Lake), as tecnologias selecionadas e suas justificativas, e os resultados de uma análise exploratória preliminar conduzida sobre 45.061 registros reais coletados do INMET e do IQAir entre janeiro de 2025 e março de 2026.

Palavras-chave: Engenharia de Dados. Qualidade do Ar. Arquitetura Medalhão. Delta Lake. Apache Spark. Brasília. Motociclistas.

SUMÁRIO

1 DESCRIÇÃO DO PROJETO4

2 DEFINIÇÃO E CLASSIFICAÇÃO DOS DADOS6

3 DOMÍNIOS E SERVIÇOS8

4 ARQUITETURA — FLUXO DE DADOS10

5 TECNOLOGIAS — COMO SERÁ FEITO13

6 ANÁLISE EXPLORATÓRIA DE DADOS (EDA)16

7 CONSIDERAÇÕES FINAIS19

8 REFERÊNCIAS21

1 DESCRIÇÃO DO PROJETO

1.1 Nome e Contexto de Negócio

O projeto recebe o nome de MotoAR — uma combinação das palavras Moto e AR (qualidade do ar), sinalizando seu propósito central. O contexto de negócio está inserido na intersecção entre mobilidade urbana, saúde pública e engenharia de dados aplicada.

Brasília é uma das cidades com maior frota de motocicletas do Brasil, com estimativa de mais de 500 mil veículos. Motociclistas ficam diretamente expostos ao ar externo durante toda a sua jornada, sem qualquer barreira de filtração como a que existe em veículos fechados. Em período de estiagem — que na capital federal pode durar de quatro a seis meses — os índices de PM2.5 (material particulado fino) podem ultrapassar em cinco vezes os valores da estação chuvosa, frequentemente superando os limites recomendados pela Organização Mundial da Saúde (OMS).

Apesar desse risco concreto e mensurável, não existe nenhuma ferramenta pública, acessível e automatizada que integre dados de sensores de qualidade do ar com recomendações práticas de equipamento de proteção para motociclistas. O MotoAR pretende preencher essa lacuna.

1.2 Problema e Objetivos

O problema central que o MotoAR pretende resolver pode ser enunciado da seguinte forma: motociclistas em Brasília não têm acesso fácil a informações integradas sobre qualidade do ar que permitam decisões de proteção antes de iniciar uma viagem.

Os objetivos principais do projeto são:

- Coletar dados de qualidade do ar e meteorologia de múltiplas fontes abertas e de baixo custo (IQAir, INMET, Open-Meteo);
- Armazenar, processar e transformar esses dados em um pipeline robusto, confiável e rastreável;
- Treinar um modelo preditivo capaz de gerar um score de qualidade do ar para as próximas 6 horas, específico para Brasília;
- Entregar recomendações práticas e acionáveis ao motociclista: qual equipamento usar, se há alerta de queimada, se há inversão térmica noturna.

1.3 Stakeholders e Usuários Finais

Os principais stakeholders do projeto são:

- Motociclistas urbanos: usuário final primário, que consome as recomendações via aplicativo ou notificação;
- Pesquisadores e autoridades de saúde pública: interessados nos dados históricos e no painel analítico de exposição por região;
- Equipe do projeto: cientistas de dados responsáveis pelo pipeline, pelo modelo e pela manutenção da plataforma;
- Professor e instituição: avaliadores do planejamento e da implementação do ciclo de vida de engenharia de dados.

2 DEFINIÇÃO E CLASSIFICAÇÃO DOS DADOS

2.1 Fontes de Dados

O projeto MotoAR utiliza dados provenientes de três fontes externas abertas, todas acessíveis por APIs REST públicas ou gratuitas para uso acadêmico. A Tabela 1 apresenta o inventário completo das fontes com seus respectivos metadados.

Fonte	Cobertura	Registros	Parâmetros	Formato	Frequência	Classificação
INMET — CRAS Fercal	Jan–Dez 2025	8.760	14	CSV/JSON	Horária (1h)	Batch / Operacional
INMET — Escola	Jan–Dez 2025	8.760	8	CSV/JSON	Horária (1h)	Batch / Operacional
IQAir — Brasília	Mar 2026	9.853	5	JSON (REST)	~1 minuto	Near Real-Time
IQAir — Escola 115	Mar 2026	8.766	5	JSON (REST)	~1 minuto	Near Real-Time
IQAir — UnB Gama	Mar 2026	9.053	5	JSON (REST)	~1 minuto	Near Real-Time
IQAir — Finatec	Mar 2026	8.629	5	JSON (REST)	~1 minuto	Near Real-Time
Open-Meteo	Futuro	—	6 (forecast)	JSON (REST)	6 horas	Batch / Previsão
INMET 2024	Jan–Dez 2024	8.784	14	CSV/JSON	Horária (1h)	Batch / Histórico

Tabela 1 — Inventário de Fontes de Dados do MotoAR

O volume total de dados já coletados até a data deste relatório é de 45.061 registros. Com a coleta contínua, estimamos um crescimento de aproximadamente 17.520 novos registros por mês (soma de todas as fontes ativas), alcançando cerca de 250 mil registros ao longo de 2026. O formato principal de armazenamento final será Parquet, escolhido por sua compressão eficiente e desempenho em leituras analíticas com filtros de coluna.

2.2 Classificação dos Dados

2.2.1 Dados Operacionais (Batch e Históricos)

Os dados do INMET se enquadram na categoria de dados operacionais e históricos. São coletados uma vez por hora por estações automáticas físicas, disponibilizados via API em formato CSV ou JSON, e possuem caráter transacional: cada leitura horária é um registro imutável que representa o estado do ambiente

naquele instante. São estruturados, com schema fixo de 14 parâmetros meteorológicos (temperatura, umidade, pressão, velocidade do vento, direção do vento, radiação solar, precipitação, PM2.5, PM10, NO2, entre outros).

Os dados históricos de 2024 e 2025 já foram coletados e armazenados, perfazendo 17.520 registros das duas estações ativas. Esses dados históricos são o insumo principal para o treinamento do modelo preditivo.

2.2.2 Dados de Near-Real-Time (Quasi-Streaming)

Os dados do IQAir são coletados com frequência de aproximadamente 1 minuto por sensor — uma frequência que se aproxima de streaming, embora tecnicamente seja implementada como micro-batching (polling via API REST). Quatro sensores estão distribuídos em locais estratégicos de Brasília: Finatec (área verde/lagos), Escola 115 Norte (área residencial), Centro de Brasília e UnB Odisseia Gama (periferia). Cada registro contém AQI (Índice de Qualidade do Ar na escala EPA), PM2.5, temperatura, umidade e velocidade do vento, no formato JSON.

A principal característica desses dados é sua alta frequência e a necessidade de processamento rápido para que as recomendações ao motociclista estejam sempre atualizadas. Eventualmente, na Parte 2 do projeto, o IQAir poderá ser integrado via um tópico Kafka para aproximar-se de streaming verdadeiro.

2.2.3 Dados de Previsão

O Open-Meteo fornece previsões meteorológicas para as próximas 6 horas (temperatura, precipitação, umidade, vento). Esses dados são consumidos em batch, horariamente, e alimentam o modelo preditivo como features de contexto futuro, permitindo que o score MotoAR seja gerado não apenas para o momento atual, mas para as próximas horas.

2.3 Qualidade e Limpeza dos Dados

Uma prova de conceito do pipeline de limpeza foi executada sobre os dados do IQAir, conforme detalhado no pipeline_report.json gerado automaticamente pelo módulo motoar_pipeline.py. De 36.301 registros brutos, foram removidos 238 (0,66%) por categoria inválida (status 'não encontrado'). Nenhum valor fora dos intervalos aceitáveis foi detectado (AQI 0–500, PM2.5 0–500, temperatura -10°C a 60°C, umidade 0–100%). Adicionalmente, 32.255 registros foram sinalizados com

flag sensor_frozen=1 para indicação de possível sensor travado, o que não implica remoção imediata, mas geração de alerta.

3 DOMÍNIOS E SERVIÇOS

3.1 Identificação dos Domínios de Negócio

O MotoAR foi modelado segundo os princípios de Domain-Driven Design (DDD), organizando as responsabilidades em cinco domínios de negócio bem delimitados. Essa separação garante baixo acoplamento entre os módulos, alta coesão interna em cada domínio, e facilidade de evolução independente — por exemplo, trocar o modelo preditivo não afeta o domínio de ingestão.

O Diagrama 1 a seguir representa graficamente a organização dos domínios, seus serviços internos e as tecnologias associadas a cada um.

MOTOAR — DOMÍNIOS DE NEGÓCIO				
Domínio Ingestão	**Domínio Processamento**	**Domínio Feature Eng.**	**Domínio Modelo ML**	**Domínio Entrega**
• IQAir API • INMET API • Open-Meteo	• Validação • Outlier Clip • Deduplicação	• 86 features • Lag/Rolling • Temporais	• XGBoost • Score 0-100 • Classificação	• App MotoAR • Flask API • Notificações
`Python / Requests`	`Great Expect. / PySpark`	`PySpark / Delta Lake`	`Scikit-Learn / MLflow`	`Flask / Metabase`
Fluxo: Ingestão → Processamento → Feature Eng. → Modelo ML → Entrega				

Diagrama 1 — Domínios de Negócio, Serviços e Tecnologias do MotoAR

3.2 Descrição Detalhada dos Domínios

3.2.1 Domínio de Ingestão

Responsável por toda a coleta de dados externos. Gerencia os clientes de API para IQAir, INMET e Open-Meteo, controla a frequência de polling de cada fonte e realiza a primeira camada de validação (schema enforcement, filtro de status). A saída deste domínio são arquivos JSON brutos gravados no storage, sem nenhuma transformação de conteúdo. Qualquer mudança nas APIs externas impacta apenas este domínio.

3.2.2 Domínio de Processamento

Recebe os dados brutos do domínio de Ingestão e aplica regras de qualidade: remoção de outliers por clipping (PM2.5 > 500 µg/m³), imputação de nulos com

forward fill (adequado para séries temporais), detecção de sensor travado por análise de padrões históricos, e deduplicação. Os dados processados são gravados na camada Bronze do Delta Lake, onde são imutáveis e versionados.

3.2.3 Domínio de Feature Engineering

Recebe os dados da camada Bronze e Silver e constrói as 86 features que alimentam o modelo preditivo. As features são organizadas em quatro categorias: temporais cíclicas (hora e mês transformados com seno e cosseno para capturar periodicidade), lag features (valores passados de PM2.5 com delays de 1h, 3h, 6h e 24h), rolling windows (médias móveis de 3h, 6h, 12h e 24h), e indicadores derivados (chuva acumulada, delta de temperatura, flag de inversão térmica). A saída é a camada Gold do Delta Lake.

3.2.4 Domínio de Modelo Preditivo

Treina e serve o modelo de machine learning. O XGBoost é treinado sobre dados históricos Gold e gera um score MotoAR de 0 a 100 com classificação de risco (ótimo, favorável, atenção, crítico). O MLflow rastreia todos os experimentos, permitindo comparação de modelos e rollback para versões anteriores. A inferência em tempo real é realizada via Flask API REST com latência alvo abaixo de 500 ms.

3.2.5 Domínio de Entrega

Responsável por transformar o score e as features em recomendações acionáveis. Gera os 6 tipos de recomendação de equipamento (capacete com filtro, máscara N95, óculos, jaqueta pesada, jaqueta ventilada, hidratação extra), dispara alertas automáticos em caso de score Crítico e alimenta dashboards analíticos no Metabase para visualização histórica.

3.3 Serviços Compartilhados

Dois serviços operam transversalmente a todos os domínios. O Apache Airflow atua como orquestrador central, coordenando as dependências entre domínios e garantindo que a Feature Engineering só execute após o Processamento ter concluído sem erros. O Grafana + Prometheus monitora métricas de todos os serviços em tempo real, disparando alertas quando SLAs são violados ou quando qualquer domínio falha.

4 ARQUITETURA — FLUXO DE DADOS

4.1 Tipo de Arquitetura Escolhida

A arquitetura do MotoAR é baseada na Arquitetura Medalhão (Medallion Architecture), um padrão consagrado em projetos de Data Lakehouse que organiza os dados em três camadas progressivas de qualidade e granularidade: Bronze, Silver e Gold. Essa escolha foi motivada por três características do projeto que a tornam especialmente adequada.

Em primeiro lugar, a rastreabilidade: como os dados de saúde pública precisam ser auditáveis, manter os dados brutos na camada Bronze de forma imutável permite reconstruir o histórico completo a qualquer momento. Em segundo lugar, a progressividade: cada camada adiciona valor específico — Bronze é ingestão, Silver é limpeza, Gold é análise — o que facilita o depuramento de problemas. Em terceiro lugar, a flexibilidade: o Delta Lake, que sustenta todas as camadas, permite time travel, ACID transactions e schema evolution sem reescrever toda a arquitetura.

Outras arquiteturas foram consideradas e descartadas por razões específicas. A Lambda Architecture (batch layer + speed layer) foi descartada porque implica manter duas bases de código para o mesmo resultado — uma batch e uma streaming — o que eleva a complexidade de manutenção sem ganho de latência justificável para o caso de uso. A Kappa Architecture foi considerada inadequada porque o projeto não tem requisitos de latência de streaming verdadeiro (milissegundos); o micro-batching do IQAir a cada 5 minutos atende os requisitos com muito mais simplicidade. O Data Mesh foi descartado por ser excessivo para o número atual de domínios e para a equipe de três pessoas.

4.2 Diagrama de Arquitetura — Fluxo Ponta a Ponta

O Diagrama 2 apresenta o fluxo completo de dados, desde as fontes originais até a entrega ao usuário final, com indicação explícita das tecnologias em cada etapa e dos caminhos de batch e near-real-time.

ARQUITETURA MOTOAR — FLUXO PONTA A PONTA (ARQUITETURA MEDALHÃO)

▼

ORIGEM (Fontes)	▸ IQAir API — JSON, ~1 min, 4 sensores em Brasília, PM2.5 / AQI / Temperatura ▸ INMET API — CSV/JSON, horário, 14 parâmetros meteorológicos, 2 estações automáticas ▸ Open-Meteo — JSON, 6h forecast, temperatura / precipitação / vento

▼

INGESTÃO (Coleta)	▸ Python Requests + Schedule: polling periódico (batch horário + near-real-time a cada 5 min) ▸ Validação inicial: filtro status 'Ok' (INMET), schema enforcement ▸ Formato de saída: JSON bruto → gravado em MinIO (camada Bronze, imutável) ▸ Tipo de dado: batch + micro-batching (quasi-streaming)

▼

ARMAZENAMENTO BRONZE	▸ MinIO (S3-compatível) + Delta Lake: dados brutos imutáveis, versionados ▸ ACID Transactions: leituras nunca veem escrita parcial ▸ Time Travel: possível consultar estado dos dados em qualquer ponto no tempo ▸ Formato: Parquet + Delta Log (.json de transações)

▼

TRANSFORMAÇÃO SILVER	▸ Apache Spark (PySpark): limpeza pesada — deduplicação, outlier clipping, forward fill ▸ Great Expectations: validação automatizada (not_null, in_range, unique) ▸ Saída: tabelas Silver limpas em Delta Lake, prontas para feature engineering

▼

FEATURE ENG. GOLD	▸ dbt (data build tool): transformações SQL versionadas no Git ▸ 86 features geradas: temporais cíclicas (hora/mês seno-cosseno), lag features (1h/3h/6h/24h), ▸ rolling windows (médias móveis 3h/6h/12h/24h), indicadores (chuva, inversão térmica) ▸ Testes automáticos, lineage e documentação gerada automaticamente pelo dbt

▼

MODELO ML (Predição)	▸ XGBoost / Random Forest: treinado sobre dados Gold históricos ▸ Entrada: 86 features em tempo real | Saída: Score MotoAR 0–100 + classe de risco ▸ MLflow: rastreamento de experimentos, versionamento de modelos ▸ Inferência: Flask API REST expõe endpoint /predict (latência alvo < 500 ms)

▼

ORQUESTRAÇÃO	▸ Apache Airflow: DAGs em Python — pipelines batch diários ▸ Retry automático com backoff exponencial em caso de falha ▸ Grafana + Prometheus: alertas de anomalia e SLA violation em tempo real

▼

ENTREGA (Consumo)	▸ App MotoAR (Flask + HTML/JS): score visível + 6 tipos de recomendação de equipamento

	▸ Metabase: dashboards analíticos sobre dados Gold (histórico de poluição, sazonalidade) ▸ Push Notifications: alerta automático se score Crítico (queimada / inversão térmica)

Diagrama 2 — Arquitetura Medalhão MotoAR: Fluxo Ponta a Ponta

4.3 Caminhos Batch e Near-Real-Time

O pipeline batch é acionado pelo Airflow diariamente às 00h00, processando os registros do INMET do dia anterior, atualizando as features Gold e retreinando o modelo se novos dados históricos justificarem. Esse caminho é o responsável pela qualidade de longo prazo do modelo.

O caminho near-real-time opera em paralelo: a cada 5 minutos, um job Python faz o polling do IQAir, valida os dados recebidos e os grava na camada Bronze. Um segundo job Spark consome esses dados novos, aplica as features em modo incremental e envia o input ao modelo Flask para geração do score atualizado. Esse score é exibido imediatamente no app.

4.4 Trade-offs da Arquitetura

A Tabela 2 apresenta os principais trade-offs identificados na escolha da arquitetura Medalhão com Delta Lake, avaliando as dimensões de acoplamento, escalabilidade, disponibilidade, confiabilidade e reversibilidade.

Dimensão	Vantagem / Decisão	Risco / Limitação
Acoplamento	Domínios independentes (DDD) — mudança em um domínio não impacta outros	Requer contratos claros de dados entre domínios
Escalabilidade	MinIO + Spark escalam horizontalmente; Delta Lake suporta crescimento de volume	Ambiente local tem limite de RAM/CPU para Spark local mode
Disponibilidade	Retry automático no Airflow; Delta Lake evita dados corrompidos em falha de escrita	APIs externas (IQAir, INMET) sem garantia de SLA 99,9%
Confiabilidade	ACID via Delta Lake + Great Expectations bloqueiam dados inválidos	Open-Meteo forecast pode ter desvio em eventos extremos
Reversibilidade	Time travel (Delta Lake) permite rollback a qualquer versão; dbt versionado no Git	Reprocessamento de grandes volumes pode ser lento em hardware limitado

Tabela 2 — Trade-offs da Arquitetura Medalhão MotoAR

O principal risco arquitetural identificado é a dependência de três APIs externas sem garantia de SLA. Para mitigar isso, o pipeline implementa retry automático com backoff exponencial no Airflow e um mecanismo de cache que serve os dados da última leitura válida em caso de indisponibilidade temporária das APIs.

5 TECNOLOGIAS — COMO SERÁ FEITO

5.1 Visão Geral das Escolhas

A seleção de tecnologias do MotoAR foi orientada por dois critérios principais: compatibilidade com o ambiente local de desenvolvimento (máquinas com 8–16 GB de RAM, sem GPU dedicada) e alinhamento com o mercado de trabalho de engenharia de dados. Todas as ferramentas escolhidas são open-source, o que elimina custos de licença e garante flexibilidade para migração futura para ambientes de nuvem (AWS, GCP ou Azure) sem reescrita do código de negócio.

Etapa	Tecnologia	Justificativa
Ingestão	Python Requests + Schedule	Simplicidade e controle sobre o polling de APIs REST. Sem overhead de ferramentas pesadas para o volume atual (~45 mil registros).
Armazenamento	MinIO + Delta Lake	MinIO é 100% S3-compatível, sem vendor lock-in, rodando em Docker local. Delta Lake adiciona ACID, time travel e versionamento sobre Parquet.
Processamento	Apache Spark (PySpark)	Processamento distribuído escalável. Suporte nativo ao Delta Lake (MERGE, upsert). Melhor que Pandas para operações em escala (joins, rolling windows).
Transformação SQL	dbt (data build tool)	Transformações SQL versionadas no Git, testes automáticos (not null, unique, referential integrity) e lineage completo.
Qualidade de Dados	Great Expectations	Validação automatizada antes de cada etapa do pipeline. Bloqueia ingestão de dados corrompidos.
Orquestração	Apache Airflow	DAGs em Python para pipelines batch. Retry automático, interface web para monitoramento e maior ecossistema de operadores.
Modelo ML	XGBoost + MLflow	XGBoost é robusto para dados tabulares com features temporais. MLflow rastreia experimentos e versões do modelo.
Servir Dados	Flask API + Metabase	Flask expõe endpoint REST /predict com latência alvo <500ms. Metabase oferece dashboards analíticos sem necessidade de código.
Monitoramento	Grafana + Prometheus	Alertas em tempo real para anomalias de dados e violações de SLA. Integração nativa com Airflow e Spark.

Tabela 3 — Tecnologias por Etapa do Ciclo de Vida

5.2 Justificativas Detalhadas

5.2.1 Delta Lake vs. Parquet Puro

A escolha do Delta Lake em detrimento do Parquet puro resolve um problema crítico de confiabilidade: em Parquet sem ACID, se o Apache Spark falhar no meio de uma escrita de 10 GB, a próxima leitura encontrará dados parcialmente gravados — corrompidos do ponto de vista da aplicação. O Delta Lake resolve isso com transações ACID: uma escrita é completamente aplicada ou completamente descartada. Para um sistema de saúde que será consumido por motociclistas, garantir que o score nunca seja calculado sobre dados parciais é requisito não negociável.

Adicionalmente, o Delta Lake oferece time travel — a capacidade de consultar o estado exato dos dados em qualquer ponto do passado com um simples parâmetro de timestamp ou número de versão. Isso é essencial para o processo de auditoria de qualidade de dados e para o retreinamento do modelo com snapshots históricos específicos.

5.2.2 dbt vs. Spark Puro para Toda a Transformação

O dbt (data build tool) e o Apache Spark são complementares, não concorrentes, no pipeline do MotoAR. O Spark é usado para operações que requerem processamento distribuído e escala: deduplicação em grandes volumes, operações de MERGE (upsert) sobre o Delta Lake, computação de rolling windows sobre séries temporais. O dbt é usado para as transformações analíticas de Silver para Gold: construção das features de negócio, aplicação de regras de qualidade como testes automáticos (not null, unique, referential integrity) e geração de documentação e lineage de dados automaticamente a partir dos modelos SQL.

A separação de responsabilidades entre os dois evita o problema comum de ter um código PySpark monolítico e difícil de testar. Cada modelo dbt é um arquivo SQL independente, testado, versionado no Git e com documentação gerada automaticamente — uma prática de DataOps que seria inviável de replicar apenas com PySpark.

5.2.3 MinIO vs. AWS S3

O MinIO foi escolhido como substituto local do AWS S3 por ser 100% compatível com a API S3. Isso significa que todo código que conecta ao MinIO — PySpark, dbt, Airflow, Great Expectations — conectará ao S3 real em produção com

uma única mudança de endpoint e credenciais. Não há reescrita de código. Esse princípio de portabilidade zero-custo é fundamental para um projeto acadêmico que pode eventualmente escalar para produção real.

5.2.4 Apache Airflow vs. Prefect / Dagster

O Apache Airflow foi escolhido como orquestrador por possuir o maior ecossistema de operadores pré-construídos do mercado: SparkSubmitOperator, DbtOperator e AirbyteTriggerSyncOperator são todos disponíveis nativamente. Dagster e Prefect são alternativas modernas e interessantes, mas com curvas de aprendizado maiores e ecossistemas menores de operadores para as ferramentas específicas do projeto. O Airflow, apesar de ter uma configuração inicial mais verbosa, oferece a maior garantia de que todos os conectores necessários existirão prontos.

5.3 Considerações de DataOps e Governança

A governança de dados do MotoAR é implementada em três camadas. A camada de qualidade usa o Great Expectations para validação automática antes de cada etapa do pipeline, com bloqueio explícito caso qualquer expectativa falhe. A camada de rastreabilidade usa o Delta Lake para time travel e o dbt para lineage completo entre tabelas. A camada de monitoramento usa o Grafana para alertas em tempo real de anomalias de dados (por exemplo, PM2.5 médio da última hora acima de 3 desvios-padrão da média histórica).

A segurança dos dados é garantida pelo fato de que nenhum dado pessoal de usuário é coletado na Parte 1 — todos os dados são de sensores ambientais e APIs públicas. Na Parte 2, quando o app começar a coletar histórico de uso individual, será necessário implementar criptografia em repouso no MinIO e controle de acesso por perfil.

6 ANÁLISE EXPLORATÓRIA DE DADOS (EDA)

6.1 Conjunto de Dados Analisado

A análise exploratória foi conduzida sobre 45.061 registros reais coletados entre janeiro de 2025 e março de 2026, oriundos de duas fontes: o INMET (estação CRAS Fercal e estação Escola, dados horários de 2025) e o IQAir (4 sensores distribuídos em Brasília, dados de março de 2026). Essa análise teve dois objetivos: validar empiricamente a relevância do problema que o MotoAR se propõe a resolver, e informar as decisões de feature engineering e de limiares para as recomendações.

6.2 Resultados INMET — Estação CRAS Fercal

Métrica	INMET — CRAS Fercal	IQAir (média 4 sensores)
PM2.5 médio anual	11,55 µg/m³	AQI médio: 10,2
Desvio padrão	11,91 µg/m³ (alta variab.)	—
Pico máximo	156,9 µg/m³ (out/2025)	—
Mediana (P50)	7,32 µg/m³	—
P95	35,66 µg/m³ (Ruim — OMS)	—
P99	54,44 µg/m³ (Perigoso)	—
Sensor mais poluído	—	UnB Gama: AQI 13,4
Sensor mais limpo	—	Finatec: AQI 7,8
Spread espacial	—	5,5 pontos AQI (70% maior)

Tabela 4 — Estatísticas Descritivas do PM2.5 (INMET 2025) e IQAir (março 2026)

O PM2.5 médio anual de 11,55 µg/m³ está abaixo do limite anual da OMS (15 µg/m³ para PM2.5), o que à primeira vista poderia parecer satisfatório. Contudo, o desvio padrão de 11,91 µg/m³ — quase igual à média — revela uma distribuição altamente assimétrica e com cauda longa. O pico máximo de 156,9 µg/m³, registrado em outubro de 2025 durante o auge da temporada de queimadas, é dez vezes o limite diário da OMS. O P95 de 35,66 µg/m³ classifica-se como Ruim na escala EPA, e o P99 de 54,44 µg/m³ como Perigoso.

O padrão de sazonalidade é marcante: o PM2.5 médio na estação seca é 5,3 vezes maior do que na estação chuvosa (22,3 µg/m³ em setembro versus 4,2 µg/m³ em maio). Esse fator sazonal justifica a inclusão de features cíclicas de mês no

modelo preditivo, uma vez que o modelo precisa aprender que setembro é estruturalmente diferente de maio.

6.3 Padrões Horários Identificados

A análise horária revelou um padrão de duplo pico no PM2.5 ao longo do dia, consistente em ambas as fontes de dados (INMET e IQAir). O primeiro pico ocorre entre 7h e 9h, associado ao tráfego matutino intenso, com PM2.5 médio de 13 µg/m³. O segundo pico ocorre entre 19h e 21h, com PM2.5 médio de 18 µg/m³ na estação seca, atribuído ao fenômeno de inversão térmica noturna — quando a camada mais fria de ar próxima ao solo impede a dispersão dos poluentes.

Esse padrão de duplo pico justifica a inclusão de features temporais cíclicas (seno e cosseno da hora do dia) como inputs do modelo. Uma feature linear de hora (0 a 23) não capturaria adequadamente a periodicidade circular do fenômeno — por exemplo, a semelhança entre as 23h e as 0h.

O efeito da chuva também foi quantificado: dias com precipitação acumulada superior a 5 mm apresentam redução média de 34% no PM2.5 em relação a dias secos, pelo mecanismo de lavagem da atmosfera. Essa descoberta originou a feature de chuva acumulada nas últimas 6 horas, incluída no conjunto de features Gold.

6.4 Heterogeneidade Espacial — IQAir

Um dos achados mais importantes da EDA foi a significativa variação espacial entre os quatro sensores IQAir. O sensor da UnB Odisseia Gama, localizado em região periférica, apresentou AQI médio de 13,4, enquanto o sensor do Finatec, próximo a lagos e áreas verdes, apresentou AQI médio de 7,8. O spread de 5,5 pontos de AQI implica que motociclistas em regiões periféricas de Brasília têm exposição a poluentes cerca de 70% maior do que motociclistas em regiões nobres — um dado relevante do ponto de vista de justiça ambiental e que indica a necessidade de expandir a cobertura de sensores para regiões com mais motociclistas.

6.5 Correlações e Implicações para o Modelo

A correlação entre PM2.5 e NO2 foi de r = 0,47, um valor moderado que indica que os dois poluentes compartilham fontes comuns (tráfego e queimadas)

mas também têm dinâmicas independentes. Na estação seca, o NO2 explica até 60% da variância do PM2.5, o que sugere que a queimada tem componente nitrogenado relevante. Essa correlação justifica a inclusão do NO2 como feature de input no modelo, mesmo que o alvo de predição seja PM2.5.

7 CONSIDERAÇÕES FINAIS

7.1 Síntese do Planejamento

O MotoAR foi planejado como um ciclo de vida completo de engenharia de dados, com foco em um problema real e relevante para a mobilidade urbana de Brasília. A escolha da Arquitetura Medalhão com Delta Lake, Apache Spark e dbt representa um stack tecnológico moderno, alinhado com as melhores práticas do mercado, e ao mesmo tempo viável para implementação local com recursos computacionais limitados.

Os cinco domínios de negócio — Ingestão, Processamento, Feature Engineering, Modelo ML e Entrega — foram desenhados com baixo acoplamento e responsabilidades claras, permitindo que cada componente evolua de forma independente. A análise exploratória preliminar validou empiricamente que o problema existe, que os dados disponíveis têm qualidade suficiente para suportá-lo, e que os padrões identificados são consistentes entre as duas fontes de dados utilizadas.

7.2 Principais Riscos e Limitações

- Disponibilidade de APIs externas: IQAir, INMET e Open-Meteo são dependências externas sem garantia de SLA. A estratégia de mitigação é o cache da última leitura válida e retry automático no Airflow.
- Limitações de hardware: Apache Spark em modo local consome até 8 GB de RAM para os volumes atuais. Em máquinas com menos de 8 GB, o modo local pode ser substituído por pandas para volumes pequenos, sem perda funcional na Parte 2.
- Cobertura geográfica: quatro sensores IQAir cobrem Brasília de forma parcial. Regiões com maior densidade de motociclistas (cidades satélites) ainda não têm cobertura adequada.
- Representatividade temporal do modelo: o modelo será treinado com dados de 2025. Eventos climáticos atípicos em 2026 (El Niño, mudanças de ventos predominantes) podem degradar a performance preditiva.

7.3 Próximos Passos — Parte 2 (Implementação)

A Parte 2 do projeto consistirá na implementação completa da arquitetura planejada, seguindo um roteiro de seis semanas:

- Semana 1 — Infraestrutura Base: Docker Compose com MinIO, Spark, PostgreSQL, Airflow e Grafana operacionais, com dados sintéticos gerados para testes.

- Semana 2 — Ingestão: clientes de API para IQAir, INMET e Open-Meteo com validação de schema e gravação na camada Bronze do Delta Lake.
- Semana 2–3 — Processamento: Spark jobs para limpeza Bronze → Silver, Great Expectations para validação automática.
- Semana 3 — Transformação: modelos dbt para Silver → Gold com testes automáticos e lineage completo.
- Semana 4 — Orquestração: DAGs Airflow para o pipeline batch diário com retry e alertas Grafana.
- Semana 4 — Consumo: Metabase dashboards sobre dados Gold, Flask API com inferência do modelo XGBoost.

O critério de sucesso da Parte 2 será: pipeline batch rodando diariamente sem erros, dados Gold 100% testados e documentados, cinco ou mais dashboards no Metabase utilizáveis, Flask API retornando predições com latência abaixo de 500 ms, e Grafana detectando anomalias em tempo real.